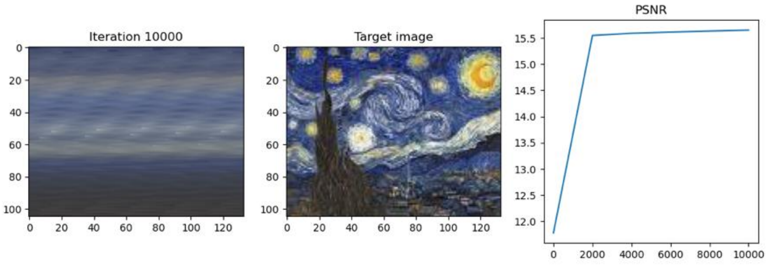
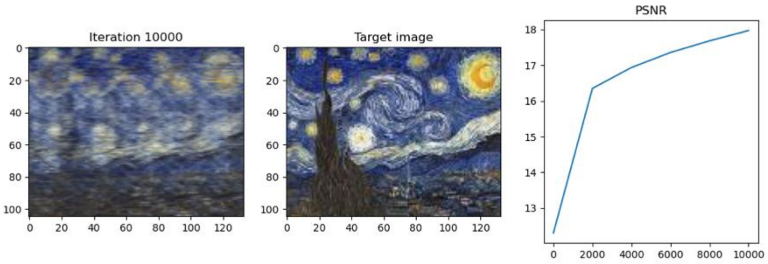
Part 1: 2D image representation

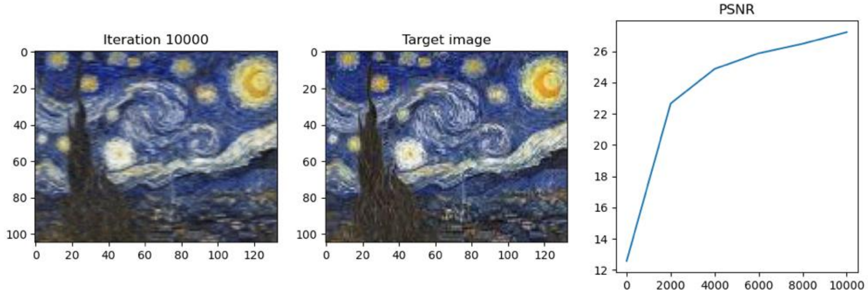
0



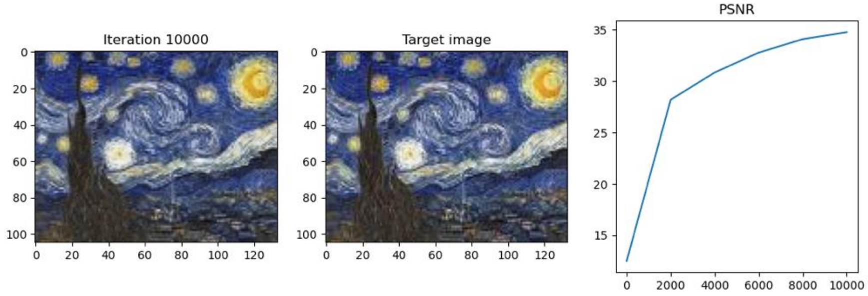
2



4



6



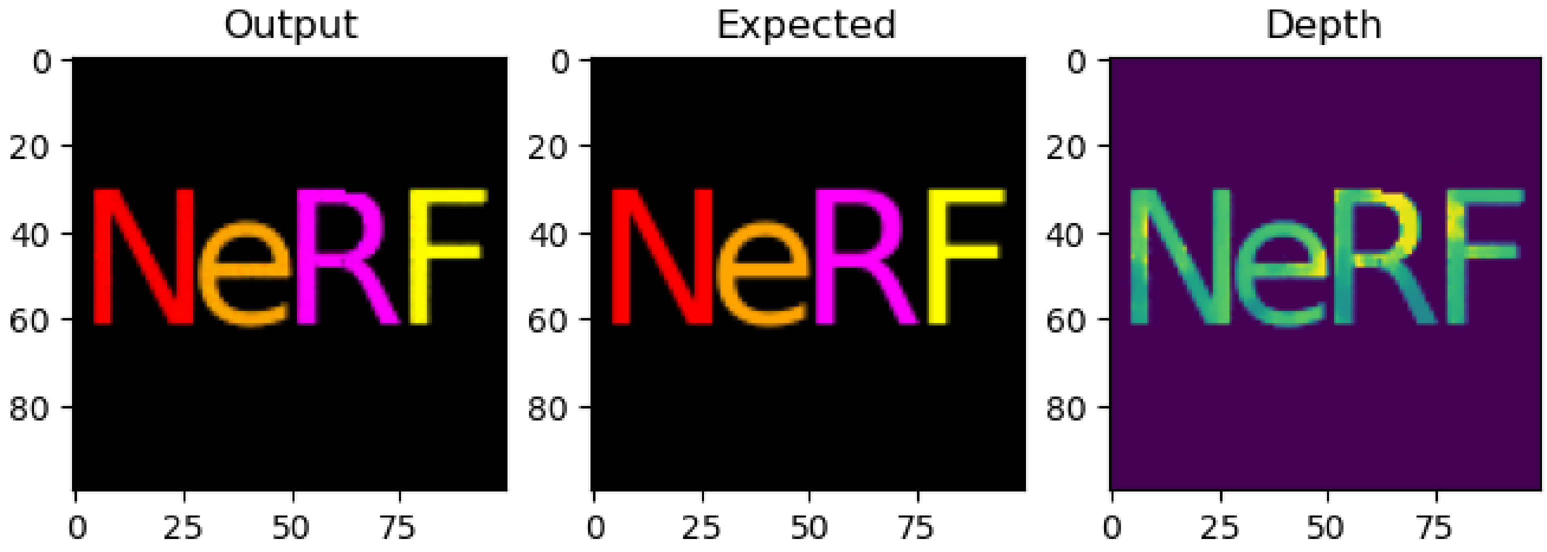
Part 1: 2D image representation

Explain the impact of positional encoding and how varying the number of frequencies affects the results.

Without positional encoding, the network has spectral bias, and it can learn low-frequency features, producing a blurry image with a PSNR of ~15.5. The reconstructed image doesn’t capture any fine details.
As the number of frequencies increases, the network gains the ability to represent higher-frequency signals. At num_frequencies=2, the broad color regions and rough shapes can be visible. At num_frequencies=4, finer details can be visible. At num_frequencies=6, the reconstruction matches the target, capturing fine textures and sharp color transitions, getting the highest PSNR of ~34.

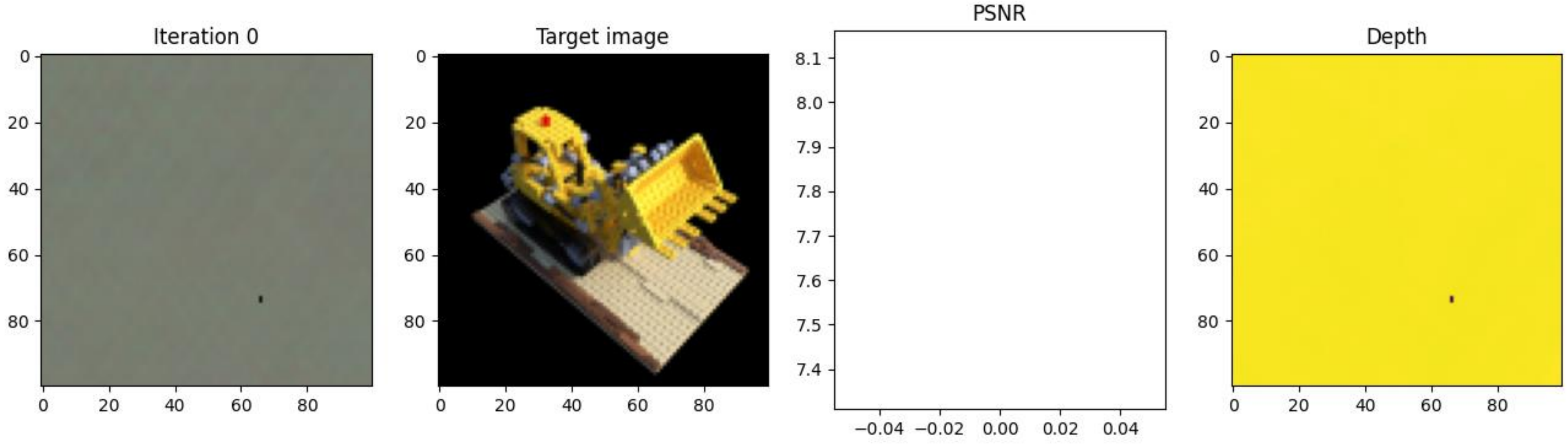
Part 2: Rendering Single Image with NeRF

Paste the result of running render_image_test() in part 2(e), showing the output from your rendering code



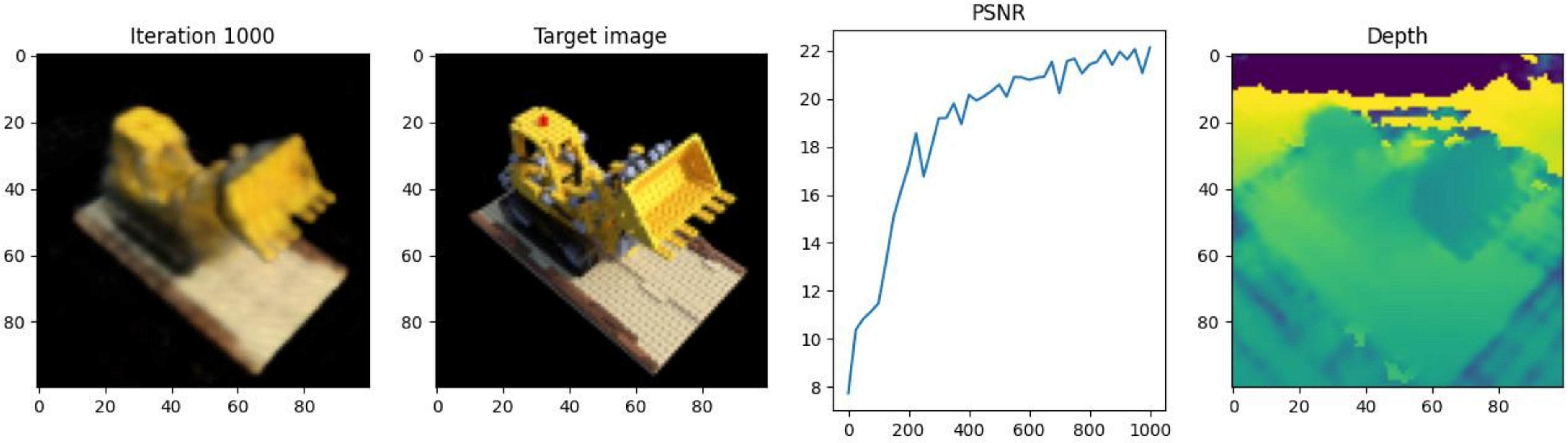
Part 3: Training NeRF

Paste your outputs of the rendered RGB image and its depth map after one iteration of NeRF



Part 3: Training NeRF

Paste your outputs of the rendered RGB image and its depth map of the held-out view



Part 3: Training NeRF

Report your PSNR score after 1000 iterations:

Iteration 1000 | Loss: 0.0061 | PSNR: 22.13 | Time per iter: 0.68s | Total time: 11.36 min

Bells & Whistles (Additional EC)

[Add any additional extra credit or experiments you did here]